

BASAL, RAFFY

```
# Creating table into database!!!
import sqlite3
from datetime import datetime
# Connect to sqlite database
conn = sqlite3.connect('banking.db')
# cursor object
cursor = conn.cursor()

# code tells the SQLite database to enforce foreign key constraints
cursor.execute("PRAGMA foreign_keys = ON")

# create table User
cursor.execute("""
CREATE TABLE IF NOT EXISTS User (
    customer_name TEXT,
    account_number INTEGER PRIMARY KEY,
    address TEXT,
    phone TEXT
)
""")

# create table Account
cursor.execute("""
CREATE TABLE IF NOT EXISTS Account (
    accNo INTEGER PRIMARY KEY,
    custName TEXT,
    balance REAL,
    FOREIGN KEY (accNo) REFERENCES User(account_number)
)
""")

# create table Bank
cursor.execute("""
CREATE TABLE IF NOT EXISTS Bank (
    custDetails INTEGER,
    loanDetails TEXT,
    txNo INTEGER PRIMARY KEY AUTOINCREMENT,
    txDate TEXT,
    txTime TEXT,
    FOREIGN KEY (custDetails) REFERENCES User(account_number)
)
""")

conn.commit()

# Helper Functions - showing datetime everytime a transaction is done.
```

```

def current_timestamp():
    now = datetime.now()
    return now.strftime("%Y-%m-%d"), now.strftime("%H:%M")

# Classes
class User:
    def __init__(self, name, account_number, address, phone):
        self.name = name
        self.account_number = account_number
        self.address = address
        self.phone = phone

    def create_account(self, amount):
        # Check if the account already exists
        cursor.execute("SELECT 1 FROM User WHERE account_number = ?",
            (self.account_number,))
        account_exists = cursor.fetchone() # display the specified data.

        if not account_exists: # Only create if account doesn't exist
            cursor.execute("INSERT INTO User VALUES (?, ?, ?, ?)",
                (self.name, self.account_number,
self.address, self.phone))
            cursor.execute("INSERT INTO Account VALUES (?, ?, ?)",
                (self.account_number, self.name, amount))
            conn.commit()
            print(f"Account created for {self.name} with a deposit
{amount}.")
        else:
            print(f"Account {self.account_number} already exists.")

    def check_balance(self):
        cursor.execute("SELECT balance FROM Account WHERE accNo = ?",
            (self.account_number,))
        balance = cursor.fetchone()[0] # get the first element of the tuple, w/c is the actual account balance
        print(f"Current balance: {balance}")

class Bank:
    def __init__(self, user):
        self.user = user
        self.name = user.name
        self.account_number = user.account_number
        self.address = user.address
        self.phone = user.phone

    def give_loan(self, account_number, loan_amount):
        cursor.execute("SELECT balance FROM Account WHERE accNo = ?",
            (self.account_number,))
        account = cursor.fetchone()

```

```

        if account:
            # Record the loan in the Bank table
            cursor.execute("INSERT INTO Bank (custDetails,
loanDetails, txDate, txTime) VALUES (?, ?, ?, ?)",
                           (self.account_number, f"Loan granted:
{loan_amount}", *current_timestamp()))
            conn.commit()
            print(f"Loan of {loan_amount} granted to account
{self.account_number}.")
        else:
            print("Account not found.")

    def update_details(self, account_number, new_name=None,
new_address=None, new_phone=None):
        if new_name:
            cursor.execute("UPDATE User SET customer_name = ? WHERE
account_number = ?", (new_name, self.account_number))
        if new_address:
            cursor.execute("UPDATE User SET address = ? WHERE
account_number = ?", (new_address, self.account_number))
        if new_phone:
            cursor.execute("UPDATE User SET phone = ? WHERE
account_number = ?", (new_phone, self.account_number))
            conn.commit()
            print(f"Details updated for account {self.account_number}.")

    def collect_money(self, account_number, amount):
        cursor.execute("SELECT balance FROM Account WHERE accNo = ?",
(self.account_number,))
        balance = cursor.fetchone()
        if balance and balance[0] >= amount: # balance - check if
there if balance are found or none, balance [0] will display the
balance based on time stamp
            new_balance = balance[0] - amount
            cursor.execute("UPDATE Account SET balance = ? WHERE accNo
= ?",
                           (new_balance, self.account_number))
            cursor.execute("INSERT INTO Bank (custDetails,
loanDetails, txDate, txTime) VALUES (?, ?, ?, ?)",
                           (account_number, f"Money collected:
{amount}", *current_timestamp()))
            conn.commit()
            print(f"Collected {amount} from account
{self.account_number}. New balance: {new_balance}")
        else: # if balance did not is not more that or equal to amount
            print("Insufficient funds or account not found.")

    def make_transaction(self, account_number, description):
        cursor.execute("INSERT INTO Bank (custDetails, loanDetails,

```

```

txDate, txTime) VALUES (?, ?, ?, ?)",
                        (self.account_number, description,
*current_timestamp()))
        conn.commit()
        print(f"Transaction recorded: {description}")

if __name__ == "__main__":
    # Create a user
    user1 = User("John Doe", 101, "123 Elm St", "555-1234")
    user2 = User("John Smith", 102, "124 Shibuya", "444-4321")

    # Call create_account method on the bank object
    user1.create_account(100)    #create an object and create the
account
    user2.create_account(1000)    #create an object and create the
account

    # User operations
    user1.check_balance()

    # Bank operations
    # Create a Bank object associated with the user
    banks = Bank(user1)
    bank1 = Bank(user2)
    banks.give_loan(101, 200)    # Loan of 200 granted to account 101
    bank1.make_transaction(101, "Loaned 200")
    banks.update_details(101, new_name="John A. Doe", new_address="456
Maple St")
    banks.collect_money(101, 100)

    # Check balance again
    user1.check_balance()
    user2.check_balance()

```

```

Account 101 already exists.
Account 102 already exists.
Current balance: 100.0
Loan of 200 granted to account 101.
Transaction recorded: Loaned 200
Details updated for account 101.
Collected 100 from account 101. New balance: 0.0
Current balance: 0.0
Current balance: 1000.0

```